RIGA TECHNICAL UNIVERSITY

Faculty of Computer Science, Information Technology and Energy

Computer Science

Nihad Guliyev, Rashad Gasimov, Laurick Spahn-Eigner, Kristen Couty

Academic Bachelor Level Study Program Computer Science

Student ID No 251ADB196, 251ADB248, 260ADB075, 260ADB074

UniConnect

BACHELOR THESIS: 2nd SEMESTER REPORT

Scientific adviser <scientific degree, academic position>

<Name, surname>

ABSTRACT

EDUCATIONAL TECHNOLOGY, REAL-TIME COMMUNICATION, WEB ARCHITECTURE, ROLE-BASED ACCESS

The rapid digitalization of higher education has blurred the lines between personal and academic communication, often forcing students and faculty to rely on fragmented, generic messaging platforms that lack structured organizational tools. The goal of this bachelor thesis is to design, develop, and deploy "UniConnect", a dedicated, centralized web platform tailored specifically for university class communication and organization. The research addresses the inefficiencies of current decentralized methods by integrating real-time group messaging, official announcement broadcasting, interactive timetables, and a secure polling system into a unified environment. A lightweight, containerized solution was implemented using a Python FastAPI backend with WebSockets and a zero-dependency Vanilla JavaScript frontend. The resulting system successfully provides a secure, role-based Digital Learning Environment that streamlines academic cohort management and improves communication efficiency without relying on third-party commercial applications.

The bachelor thesis consists of [X] pages, it contains [X] figures, [X] tables, [X] appendixes, and [X] sources of reference.

ANOTĀCIJA

IZGLĪTĪBAS TEHNOLOĢIJAS, REĀLLAIKA KOMUNIKĀCIJA, TĪMEKĻA ARHITEKTŪRA, LOMĀS BALSTĪTA PIEKĻUVE

Straujā augstākās izglītības digitalizācija ir izpludinājusi robežas starp personīgo un akadēmisko komunikāciju, bieži piespiežot studentus un mācībspēkus paļauties uz fragmentētām, vispārīgām ziņojumapmaiņas platformām, kurām trūkst strukturētu organizatorisko rīku. Šī bakalaura darba mērķis ir projektēt, izstrādāt un ieviest "UniConnect" – specializētu, centralizētu tīmekļa platformu, kas pielāgota tieši universitātes grupu komunikācijai un organizācijai. Pētījums risina pašreizējo decentralizēto metožu neefektivitāti, integrējot reāllaika grupu ziņojumapmaiņu, oficiālu paziņojumu apraidi, interaktīvus sarakstus un drošu balsošanas sistēmu vienotā vidē. Tika ieviests viegls, konteinerizēts risinājums, izmantojot Python FastAPI aizmugursistēmu (backend) ar WebSockets un no atkarībām brīvu Vanilla JavaScript priekšgalsistēmu (frontend). Izveidotā sistēma veiksmīgi nodrošina drošu, lomās balstītu digitālo mācību vidi, kas racionalizē akadēmisko grupu pārvaldību un uzlabo komunikācijas efektivitāti, nepaļaujoties uz trešo pušu komerciālām lietojumprogrammām.

Bakalaura darbs sastāv no [X] lappusēm, tas satur [X] attēlus, [X] tabulas, [X] pielikumus un [X] atsauces uz informācijas avotiem.

TABLE OF CONTENTS

INTRODUCTION

Context and problems

In the modern academic environment, universities rely on robust Learning Management Systems (LMS) and official email networks for formal administration and resource distribution. While these tools are essential for the structural backbone of education, they are often too rigid and asynchronous for the dynamic, day-to-day communication required within specific educational groups, such as a class cohort or a project team.

Consequently, students, class delegates, and teachers frequently resort to informal, third-party messaging applications like WhatsApp or Discord to ask quick questions, organize study sessions, or share immediate updates. This ad-hoc approach creates several critical problems. First, academic communication becomes blurred with personal messaging. Second, important group decisions or official announcements are easily lost in the continuous chat feed. Third, these commercial platforms lack strict role-based access control, making it difficult to distinguish between a professor's official broadcast and a student's informal query. We address this problem by providing UniConnect, a dedicated, and academically focused space for these daily interactions. Improving the flow of coordination within educational groups.

Aim of the project

The primary aim of this project is to develop UniConnect, a dedicated web-based coordination and communication platform designed specifically to facilitate daily interactions and group management among students, elected class delegates, and teachers.

Project tasks/objectives

To systematically achieve this aim, the project is guided by several core objectives. It begins with designing a strict Role-Based Access Control (RBAC) interface to clearly separate the permissions and views of students, delegates, teachers, and administrators. This interface is powered by a secure backend system developed with Python and FastAPI, which handles user authentication, group management, and real-time data flow. A central task is implementing bidirectional communication channels via WebSockets to replace informal social media chats with structured, class-specific messaging. Additionally, the project involves building practical academic coordination tools, such as interactive timetables, dynamic event tracking, and secure polling mechanisms for democratic class decisions. These features are brought together through the construction of a lightweight, zero-dependency Single Page Application (SPA) frontend utilizing Vanilla JavaScript, HTML5, and CSS3. Ultimately, the project requires deploying the platform using modern DevOps practices specifically Docker containerization and a Caddy reverse proxy for automatic HTTPS and evaluating the system's overall functional reliability through targeted Use Case testing with a pilot group of users.

Scope and constraints

The scope of this project is defined to deliver a functional Minimum Viable Product (MVP) focused on real-time group coordination. The MVP includes secure registration, WebSocket-based messaging, event creation, polling, ressources sharing, timetables viewing and priority announcements.

Several features are explicitly out of scope: this platform does not aim to replace institutional LMS platforms, meaning grading systems and formal assignment submissions are excluded. Additionally, native mobile

applications and built-in video conferencing are excluded from this iteration.

Methods

The approach follows an iterative, Agile-inspired methodology. The process began with requirements engineering and the creation of visual prototypes to map out the user interface. Detailed Use Cases were then defined to translate these concepts into technical requirements.

Development was divided into manageable sprints. The initial phase focused on establishing the FastAPI architecture and the relational database schema. Subsequent sprints handled frontend integration and the link beetwin the frontend and the server. The project concludes with formal User Acceptance Testing (UAT) to validate that the platform effectively facilitates the intended academic communication scenarios.

Document structure

This documentation is organized into eight main chapters that trace the project's complete development lifecycle. Chapters 1 through 4 establish the theoretical background, define the core problem, and outline the project's objectives and scope. Chapter 5 details the functional and non-functional requirements. Chapters 6 and 7 present the system design architecture and its practical technical implementation. Finally, Chapter 8 covers the testing plan and results, followed by unnumbered concluding sections that summarize the project's outcomes, references, and supporting appendices.

1. BACKGROUND / CONCEPTUAL FRAMEWORK

1.1 Domain overview and context

The modern higher education ecosystem is heavily reliant on digital infrastructure. Universities universally deploy Learning Management Systems (LMS) and official email networks to handle formal administration, course delivery, and secure grading. However, alongside this formal digital infrastructure exists a parallel, highly dynamic domain: the day-to-day coordination of specific academic cohorts, such as a class of students or a project group.

In this domain, students, elected class delegates, and teaching staff require rapid, informal, and highly accessible communication channels to organize study sessions, share schedule updates, and clarify immediate doubts. Traditionally, this space has been left unmanaged by institutions, leading students to adopt generic commercial messaging platforms to fill the void. Consequently, the domain of academic cohort management currently operates in a fragmented state, caught between overly rigid institutional tools and overly casual, unstructured social media networks. Understanding this gap is essential for developing a system that successfully marries the speed of modern messaging with the structure required for academic organization.

1.2 Key concepts and definitions

To ensure clarity and consistency throughout this documentation, several key conceptual and technical terms are defined below:

- **Role-Based Access Control (RBAC):** A security paradigm where system access and permissions are strictly determined by the user's assigned role.

- **Single Page Application (SPA):** A web application architecture that interacts with the user by dynamically rewriting the current web page with new data from the web server, rather than loading entirely.
- **API / REST:** A standardized architectural style allowing communication between the client application and the server. It relies on stateless HTTP requests to interact with structured data.
- **WebSocket:** A persistent, full-duplex communication protocol over a single TCP connection, used to enable low-latency, real-time functionality.
- **JWT (JSON Web Token):** A secure, compact standard used for safely transmitting information between the client and server as a JSON object, utilized primarily for user authentication and authorization.
- **MVP (Minimum Viable Product):** A development technique in which a new product is built with only the core features sufficient to satisfy early users and validate the fundamental concept.
- **Use Case:** A specific sequence of interactions between a user (actor) and the system designed to achieve a defined, measurable goal.

1.3 Overview of existing solutions

The current landscape of academic communication relies on two distinct types of solutions, both of which exhibit significant structural flaws for cohort management.

On one end are formal institutional platforms like Moodle or Microsoft Teams. While highly secure and structured, these platforms suffer from low daily engagement for informal tasks. Moodle forums are asynchronous and often perceived as too formal for quick queries, while Microsoft Teams can be overly complex and resource-heavy for simple class coordination.

On the other end are commercial messaging applications such as WhatsApp, Telegram, or Discord. These platforms excel in usability and real-time delivery, which explains their massive adoption by students. However, they critically lack academic context. They do not enforce RBAC, meaning a professor cannot easily verify the identity of a student, and an elected delegate has no systemic authority to pin an official announcement over casual chat. Furthermore, these platforms inherently blur the line between personal digital lives and academic responsibilities, contributing to digital distraction and data privacy concerns. A structural pattern emerges from this analysis: existing solutions either prioritize structure at the expense of speed, or speed at the expense of structure.

1.4 Requirements perspective

From the analysis of this domain and existing approaches, it becomes clear what a "correct solution" must provide. A system operating in the academic coordination space must blend the real-time responsiveness of commercial messengers with the authoritative structure of an institutional tool.

Typically, systems of this type require clear, distraction-free navigation, real-time data flow for immediate messaging, and highly stable role verification to ensure that official announcements and democratic polls are trustworthy. Furthermore, usability and performance are paramount, as students expect modern web applications to respond instantly, mirroring the fluid experience of the social platforms they currently use. Building upon these domain expectations, the following chapter (Chapter 2) will precisely define the specific problem UniConnect aims to solve, formally establishing the project's goal and the criteria by which its success will be measured.

2. PROJECT GOAL AND PROBLEM DEFINITION

2.1 Context and stakeholders

Operating within higher education cohort management, this project bridges the gap for fast-paced, day-to-day class coordination that formal Learning Management Systems (LMS) cannot efficiently handle. To maintain clear communication hierarchies, stakeholders are distinctly categorized. Primary users consist of Students, who engage in discussions and voting, and Class Delegates, who actively manage the cohort by organizing events and polls. Secondary stakeholders—Professors and Teaching Assistants—rely on the platform as a direct, noise-free channel for urgent updates and resource sharing. Finally, University IT and Administration serve as indirect stakeholders; while not daily users, they benefit from the system's secure, containerized architecture, which effectively mitigates the data privacy risks associated with shadow IT.

2.2 Problem statement

Currently, daily academic coordination relies on generic messaging applications (such as WhatsApp or Discord) that lack structural hierarchy. This leads to severe information fragmentation, where important announcements from professors or delegates are easily buried under casual noise. Furthermore, the absence of strict Role-Based Access Control (RBAC) on these commercial platforms creates confusion regarding the authority of shared information. Consequently, students waste time searching for critical updates and resources. If left unresolved, this inefficiency will continue to cause digital fatigue, missed academic deadlines, and an unprofessional blurring of personal and academic digital boundaries.

2.3 Project goal

To provide university students, class delegates, and professors with a dedicated, interactive web-based platform that enables secure real-time messaging, centralized cohort scheduling, and verified democratic decision-making within a role-restricted environment, thereby eliminating information loss and establishing clear boundaries between academic organization and personal social media.

2.4 Success criteria

The project's success depends on meeting several measurable criteria. Primarily, all defined MVP use cases must be executable end-to-end without technical blockers. Furthermore, the system must demonstrate rigorous stability by maintaining resilient real-time connections, ensuring a crash-free demonstration, and gracefully managing empty or error states with clear UI feedback. Additionally, interactive multimedia elements—such as the live chat and dynamic polling animations—must be seamlessly integrated into the primary user journey. Finally, success requires comprehensive documentation and thorough verification; the final report must adhere to institutional guidelines and include formal testing records detailing the Pass/Fail status of every use case acceptance criterion.

2.5 Target audience and usage scenario

UniConnect is targeted at tech-literate university students and faculty who require frequent, reliable group coordination. In practice, the system will be accessed via desktop and laptop computers. The usage scenarios vary by time pressure: a student might log in for a quick, 30-second session on their phone to check a newly posted announcement or vote in a delegate

election. Conversely, during a collaborative project, users might keep the application open on a desktop browser for prolonged periods to utilize the real-time chat. The primary constraint is that the system requires a persistent internet connection to maintain the WebSocket state for live updates.

2.6 Assumptions and limitations

The definition of this project's goal is guided by several early assumptions and specific technical limitations. First, it is assumed that the MVP will utilize its own standalone JWT authentication system. For data persistence, the architecture relies on a local database rather than a distributed enterprise cloud solution, ensuring that the Dockerized application remains lightweight and easy to deploy for evaluation. Functionally, the platform's media capabilities are restricted to text-based communication, native audio and video conferencing features are explicitly out of scope. Finally, the user interface will be developed exclusively in English, and despite the potential future value of a mobile application, the current scope is strictly focused on delivering a robust, fully functional desktop version of the product.

2.7 Link to the next chapters

With the problem and project goal strictly defined, the subsequent chapters will operationalize this vision. Chapter 3 translates the core goal into high-level objectives and operational tasks, mapped against a sprint schedule. Chapter 4 establishes the strict scope boundaries, detailing what is explicitly included and excluded. Chapter 5 derives the functional and non-functional requirements necessary to satisfy the use cases, which then directly inform the System Design detailed in Chapter 6.

3. OBJECTIVES AND TASKS

3.1 Objectives

To achieve the primary goal of providing a secure, real-time academic coordination platform, the project is broken down into four high-level objectives:

- **Objective 1 (Design):** Establish and validate the user interface structure through manual prototyping and mutual agreement to ensure a clear, role-based navigation flow.
- **Objective 2 (Core Functionality):** Implement and verify the core Minimum Viable Product (MVP) use cases end-to-end (UC1-7).
- **Objective 3 (Interactivity & Multimedia):** Design and implement interactive user interface components and seamless multimedia elements within the main user flow to enhance user engagement and provide immediate, clear feedback.
- **Objective 4 (Quality & Documentation):** Ensure system stability through rigorous use case testing and finalize the academic documentation.

3.2 Tasks

To deliver the stated objectives, the work is divided into concrete, measurable tasks. Each task defines a specific deliverable and a clear completion indicator.

Task 1 aims to sketch manual wireframes for the main dashboards and establish a consensus on the global color scheme and layout. The expected outputs are hand-drawn sketches accompanied by clear CSS style guidelines. This task is considered complete when the UI layout is fully

prepared for HTML/CSS integration without the need for further structural debate.

Task 2 focuses on setting up the FastAPI backend, the SQLite database, and the overall project structure, laying the foundations for UC-01 through UC-07. The output will be a precise blueprint of the system's design alongside a working database. Completion is achieved when there are little to no remaining questions regarding how the use case features will be implemented, paired with a fully functional database.

Task 3.1 involves implementing the static user interface for the authentication pages, specifically targeting the UI layer of UC-01. The deliverables are the static HTML/CSS pages for the Landing, Login, and Register screens. The task is marked done when these interfaces accurately match the visual prototypes, ensure responsiveness across desktop and mobile devices, and include all necessary input fields and buttons.

Task 3.2 is dedicated to implementing the client-side authentication logic for the frontend JS layer of UC-01. This will result in Vanilla JS scripts designed for form validation and API communication. The task is complete once the client successfully validates user inputs, dispatches a JSON request to the server, securely stores the received JWT (such as in sessionStorage), and properly handles HTTP error codes by displaying appropriate messages like "Invalid password."

Task 3.3 targets the development of the secure backend authentication API for the backend layer of UC-01. The outputs consist of the FastAPI endpoints (/login and /register) along with their database integration. Success is defined when passwords are securely hashed prior to storage, valid login requests issue a signed JWT containing the user's role, and invalid login attempts are appropriately rejected with a 401 error.

Task 4.1 requires designing and implementing the chat interface for the UI layer of UC-02. The output will be the HTML/CSS structure for the Class Feed, encompassing message bubbles, a scrolling feed, and an input field. This task is finished when the chat view is visually distinct, automatically scrolls to the newest messages, and clearly separates the current user's messages from those of other participants.

Task 4.2 focuses on implementing the WebSocket client integration for the frontend JS layer of UC-02. The expected output is a Vanilla JS script capable of managing the WebSocket connection. The task is successfully completed when the client can establish a WebSocket connection using the JWT, send messages directly from the input field, and instantly inject incoming messages into the DOM without requiring a page refresh.

Task 4.3 involves building the real-time WebSocket server and ensuring message persistence for the backend layer of UC-02. The deliverables include the FastAPI WebSocket route and the corresponding database storage logic. Completion is reached when the server accepts WebSocket connections, strictly verifies the JWT, broadcasts incoming messages to all active clients within the specific class group, and reliably saves each message to the SQLite database.

Task 5.1 aims to implement the announcement creation modal and the feed display for the UI layer of UC-03. The output will feature an HTML/CSS modal for creation alongside visually distinct announcement cards within the feed. The task is considered done when the creation modal is fully accessible and pinned announcements are styled noticeably differently—such as with a unique background color or a priority badge—compared to regular messages.

Task 5.2 handles the client-side logic for announcements within the frontend JS layer of UC-03. This will produce a Vanilla JS script responsible

for role verification and modal interaction. The task is complete when the "Create Announcement" button is exclusively rendered for users possessing a Delegate or Professor role in their JWT, and when submitting the modal successfully dispatches a correct POST request to the API.

Task 5.3 focuses on developing the role-protected API endpoint for announcements within the backend layer of UC-03. The output is a secure FastAPI POST endpoint (/announcements). The task is finished when the endpoint rigorously verifies that the requester holds Delegate or Professor privileges before saving the record, correctly returning a 403 Forbidden error for standard student accounts.

Task 6.1 requires constructing the study event creation interface and the calendar list for the UI layer of UC-04. The output comprises an HTML/CSS form capturing Date, Time, and Location, paired with an event list view. Completion is achieved when the interface properly enforces input types—like date pickers—and displays upcoming events in a clean, easily readable list format.

Task 6.2 involves implementing the event scheduling client logic for the frontend JS layer of UC-04. The deliverable is a Vanilla JS script designed to handle the event form. The task is marked done when the client effectively validates that the selected date is in the future before sending the request, and subsequently dynamically updates the DOM list upon receiving a successful API response.

Task 6.3 targets the construction of the study event API and its database integration for the backend layer of UC-04. The output will be the necessary FastAPI endpoints (POST and GET /events). The task is complete when the server successfully persists new events to the database and accurately retrieves upcoming events that are specifically filtered by the user's class group.

Task 7.1 aims to design the election ballot and the multimedia success animation for the UI layer of UC-05. The outputs include the HTML/CSS voting interface and a CSS-based confetti or success animation. This task is finished when the interface clearly lists the available candidates and seamlessly triggers a smooth CSS animation once the success state is activated.

Task 7.2 is dedicated to implementing the voting interaction and state management for the frontend JS layer of UC-05. The expected output is a Vanilla JS script handling vote submission and UI locking. The task is complete when submitting a vote dispatches a POST request, triggers the designated success animation upon receiving a 200 OK response, and immediately disables the voting form to prevent multiple submissions.

Task 7.3 involves developing the secure voting API equipped with uniqueness constraints for the backend layer of UC-05. The output is a highly secure FastAPI POST endpoint (/vote). Completion is reached when the server safely registers the vote, strictly enforces database constraints to guarantee only one vote per student per election, and proactively returns a 403 error if a double-vote is attempted.

Task 8.1 requires building the administrative panel interface for the UI layer of UC-06. The output will be an HTML/CSS dashboard enabling professors to create groups and manage rosters. The task is considered done when the interface provides a clear, intuitive form for group naming alongside a functional mechanism—such as a list or text area—to easily add student emails.

Task 8.2 focuses on implementing the administrative client logic for the frontend JS layer of UC-06. The deliverable is a Vanilla JS script designed to handle group creation requests. The task is finished when the UI successfully aggregates the group name and student list, formats this data

into a JSON payload, properly handles the API response, and dynamically updates the professor's active class list.

Task 8.3 targets the development of the group management API and its corresponding database relations for the backend layer of UC-06. The outputs are the FastAPI POST endpoint (/groups) and the relational database tables linking Users to Groups. The task is complete when the server securely verifies the Professor role, generates the new group record, and successfully links multiple existing user accounts to this newly created group within the database schema.

3.3 Task-to-goal mapping

The following mapping demonstrates the traceability of the project, ensuring that every planned task directly supports a higher-level objective, which in turn delivers the main project goal defined in Chapter 2.

Objective 1 (Design) is delivered by Task 1. Objective 2 (Core Functionality) is delivered by Tasks 2, 3, 4, and 5. Objective 3 (Interactivity & Multimedia) is delivered by Task 6. Objective 4 (Quality & Documentation) is delivered by Task 7.

3.4 Prioritization

To ensure a realistic execution of the Minimum Viable Product (MVP) within strict academic time constraints, the development tasks are strategically prioritized using the MoSCoW method. The "Must-have" category encompasses the absolute core of the communication platform, specifically prioritizing Task 2 and Task 3 for the foundational design, Task 4 for secure user authentication (UC-01), and Task 5 for the real-time WebSocket chat integration (UC-02). The "Should-have" tier includes Task 6 for official announcements (UC-03) alongside Task 7 for the election module and its

multimedia success animation (UC-05), as these specific features provide the distinct academic value that differentiates UniConnect from generic messaging applications. Finally, the "Could-have" category targets the integration of complex .ics parsing for the official timetable view (UC-07) and a centralized resource-sharing system enabling professors and students to upload course materials. Should time become a limiting factor during development, the timetable feature will be gracefully simplified to a basic custom event listing (UC-04), and the comprehensive file uploading system will be deferred to a future iteration.

3.5 Milestones aligned with sprints

The development timeline spans four weeks of active coding, organized into a pre-sprint preparation phase followed by three distinct Agile-inspired sprints. Because the UI design, system architecture, and baseline API endpoints were fully established before the sprints began, this development phase focused entirely on the technical implementation of the frontend (HTML/CSS and Vanilla JS logic) and the final refinement of backend security, input validation, and data persistence.

Sprint 1, spanning one week, was dedicated to establishing core access by focusing on the complete authentication ecosystem. During this phase, the team implemented the user interface and client-side JavaScript logic for the login and registration flows (UC-01). Simultaneously, necessary backend adjustments were made to ensure secure JSON Web Token (JWT) handling and robust password hashing. The primary milestone for this sprint was successfully enabling a user to securely create an account and gain authenticated access to the platform.

Sprint 2 also lasted one week and shifted the focus entirely to real-time communication, specifically targeting the high-concurrency messaging

system. This phase required the intricate implementation of both the WebSocket client-side logic and the corresponding backend broadcast server (UC-02). The critical milestone achieved at the end of this sprint was the ability for users to reliably exchange instant messages in real-time within their designated class groups without requiring page reloads.

Sprint 3, the final and most extensive phase spanning two weeks, was dedicated to full feature integration and rigorous system testing. This sprint covered the implementation of all remaining administrative and interactive modules, including the Admin Panel for group management (UC-06), the official Announcement system (UC-03), and Study Event creation featuring .ics parsing (UC-04/07). Additionally, the team integrated the Resource sharing system for course materials and the secure Delegate Election module, which prominently featured the interactive CSS success animations (UC-05). The sprint formally concluded with rigorous functional testing of all integrated components, achieving the ultimate milestone: the complete implementation of the functional MVP alongside successful system-wide verification.

Sprint 4 and 5, were dedicated to testing and fixing the bugs we found.

3.6 Completion measurement

The overall progress and success of this project phase will not be measured by subjective estimates, but by empirical verification. Completion is evaluated by verifying that each Use Case passes its defined acceptance criteria (including handling at least one alternative/error flow per use case). Additionally, the system must demonstrate stability (no crashes during the main WebSocket chat flow), the CSS multimedia animation must render correctly upon voting, and the final documentation must accurately reflect the built system.

4. SCOPE AND CONSTRAINTS

4.1 In-scope

The Minimum Viable Product (MVP) of UniConnect delivers a centralized coordination environment focused on six core functional areas:

- **Role-Based Authentication (UC-01):** A secure authentification system.
- **Real-Time Group Messaging (UC-02):** A live chat interface powered by WebSockets within class groups, allowing instant communication.
- **Official Announcements (UC-03):** A priority broadcasting system allowing Professors and Delegates to inform on critical updates.
- **Study Event & Timetable Management (UC-04/07):** Tools to schedule custom events and view the weekly class schedule (including .ics parsing for official courses).
- **Vote System (UC-05):** A secure polling module with a multimedia success animation to confirm a vote has been cast.
- **Administrator Dashboard (UC-06):** An all in one dashboard for administrators to manage all the users and all the groups.
- **Resource Sharing System:** A centralized repository where users can upload and download course materials.

4.2 Out-of-scope

To ensure a successful delivery within the strict semester timeline, several non-essential features have been explicitly excluded from the Minimum Viable Product (MVP). Specifically, password recovery is not implemented in this initial iteration. Furthermore, comprehensive grade and examination management systems were excluded to avoid the complexity of handling extensive academic data under current time constraints. Similarly,

homework and assignment submission features were omitted, as these functions are already adequately covered by existing institutional Learning Management Systems (LMS). Advanced user profile customization has also been deprioritized, allowing the development focus to remain entirely on perfecting the core communication logic. Finally, the development of native iOS or Android mobile applications remains strictly out of scope, ensuring the system operates exclusively as a responsive, browser-based platform.

4.3 Constraints

The project is bound by several non-negotiable limitations that fundamentally influenced its final design. Primarily, strict time constraints required the system to be fully functional and documented within a highly compressed development cycle limiting the depth of secondary features. Regarding data management, the MVP is subject to storage constraints; all uploaded resources and databases are stored locally on the server hosting the API, explicitly bypassing external cloud solutions. Furthermore, device constraints dictate that the user interface is developed exclusively for desktop browsers. Finally, technological constraints mandate the use of Vanilla JavaScript for the frontend without the assistance of modern external frameworks.

4.4 Assumptions

The feasibility of the UniConnect project relies on several key assumptions, primarily concerning academic identity compliance. It is expected that students and professors will act in good faith by registering with their official academic email addresses and real names. If users were to create fake or anonymous identities, the authoritative nature of the platform would be significantly compromised; to mitigate this risk in the MVP,

administrators retain the ability to delete unrecognized accounts. Furthermore, regarding client-side integrity, the project assumes that users will interact with the platform as intended. While the backend strictly enforces core API security and role-based access checks, this educational MVP operates on the premise that users will not actively attempt to maliciously reverse-engineer, inject, or manipulate the Vanilla JavaScript frontend to exploit potential vulnerabilities. Finally, it is assumed that the target academic audience is proficient in English, as the user interface has been developed exclusively in this language for the initial release.

4.5 Boundaries by dimension

The operational scope of the project is defined by several strict boundaries. Functionally, the system is explicitly limited to communication, scheduling, voting, and resource sharing. From a user perspective, the platform enforces three distinct roles with absolutely no public guest access permitted. Regarding data boundaries, all relational information is securely managed within a database, while uploaded files are stored directly in local server directories. Finally, the device boundary restricts the application's intended use exclusively to desktop and laptop computer browsers.

4.6 Scope change rules

To prevent "scope creep" during the sprints, any requested addition to the functional scope must be documented and justified in the final report. New features are frozen after the start of Sprint 3 to allow for stabilization. Any reduction in the "Must-have" scope requires a clear rationale regarding technical blockers or time exhaustion, which must be reflected in the final testing report.

5. REQUIREMENTS

5.1 Requirements sources and traceability

The requirements defined in this chapter are derived directly from the project vision established in Chapter 2, the operational objectives in Chapter 3, and the scope boundaries set in Chapter 4. Each functional requirement is mapped to a specific Use Case (UC-xx) and UI screen to ensure that development remains strictly aligned with user needs and project goals.

5.2 Functional requirements

Functional requirements describe the specific behaviors and services the UniConnect system must provide.

ID	Requirement Description	Priority	Related UC
FR-01	The system shall allow users to authenticate using an institutional email and password.	Must	UC-01
FR-02	The system shall verify the user's role and group during the login process.	Must	UC-01
FR-03	The system should redirect the user to a waiting page if no group is assign to him.	Should	UC-01
FR-04	The system shall establish a persistent WebSocket connection for real-time group messaging.	Must	UC-02
FR-05	The system shall display basic message’s related metada.	Must	UC-02
FR-06	The system shall broadcast any sent message to all connected members of a class group	Must	UC-02

	instantly.		
FR-07	The system shall allow professors and delegates to publish announcements with a title and body.	Must	UC-03
FR-08	The system shall display the newest official announcements first.	Should	UC-03
FR-09	The system shall allow the user to create normal priority and urgent announcement	Should	UC-03
FR-10	The system shall allow users to create events (title, date, and location).	Must	UC-04
FR-11	The system shall allow the professors and delegates to create multiple choices votes.	Must	UC-05
FR-12	The system shall record a single, anonymous vote per student for a given vote.	Should	UC-05
FR-13	The system should show the vote results in some way at the end of the poll.	Must	UC-05
FR-14	The system shall allow administrators to create class groups, assign students via email and manage everything through a single dashboard.	Must	UC-06
FR-15	The system shall give the possibility to administrator to switch beetwin group dashboard easily.	Should	UC-06
FR-16	The system shall allow user to review their class schedule and custom events.	Must	UC-07
FR-17	The system shall display different type of events in different colors.	Should	UC-07
FR-	The system shall give to user the possibility	Should	UC-07

18	to view more details about the event by clicking on it.		
FR-19	The system shall allow users to upload course resources to a group.	Could	
FR-20	The system shall list shared resources and allow their download by all group members.	Could	
FR-21	The system shall allow user to switch beetwin groups easily without page reload.	Should	Global
FR-22	The system shall validate every user input to prevent empty or invalid data being proccessed as normal data.	Should	Global

5.3 Data requirements

This section specifies the structural models, fields, and constraints of the underlying relational database (managed via SQLAlchemy) required to support the application.

DR-01 (User & Group Management): The User object stores a unique identifier, unique email, hashed password, and first/last names. It requires a role attribute regulated by an Enum (ADMIN, TEACHER, DELEGATE, STUDENT), defaulting to STUDENT. The Group object, representing a class cohort, contains a unique identifier, a unique name, and an optional path to an associated schedule file (schedule_path).

DR-02 (Real-Time Messaging): To handle chat persistence, the Message object stores text content and a sent_at timestamp (defaulting to current UTC time). To maintain communication history, it requires foreign keys

explicitly linking the message to its author (user_id) and target group (group_id).

DR-03 (Official Announcements): For priority broadcasts, the Announcement object contains a title, text content, and a sent_at creation timestamp. Crucially, it features an urgent boolean flag to trigger priority UI styling, alongside foreign keys linking the announcement to its author and receiving group.

DR-04 (Study Events & Timetables): The Event object defines scheduled occurrences using a title, optional description, precise start/end datetimes, and a physical location string (with optional latitude/longitude floats). It is categorized by an Enum (STUDY, ACTIVITY, EXAM, COURSE) and requires foreign keys linking to its creator and associated group.

DR-05 (Polling System): To ensure data integrity, elections use three interconnected objects. The Poll object stores the title, an is_active boolean, and creation/expiration timestamps. The Choice object represents candidates, storing text, an optional manifesto, a photo URL, and a link to its parent poll_id. Finally, the Vote object securely records the voting action via a linked record containing the user_id, poll_id, and choice_id.

DR-06 (Resource Sharing System): The Resource object stores essential document metadata, including the title, local server path (file_path), file type, and a category Enum (LECTURE, EXERCISE, STUDENT, OTHER). This metadata is stored alongside an upload timestamp and foreign keys connecting the file to its uploader and target group.

5.4 Non-functional requirements

Non-functional requirements define the quality attributes and technical constraints of the system.

NFR-01 (Reliability): Upon loss of WebSocket connection or any other error , the system shall immediately display a prominent red banner indicating that the connection has been lost.

NFR-02 (Performance): The latency for sending and receiving a message or fetching informations shall not exceed 500ms under normal network conditions.

NFR-03 (Compatibility): The application shall be fully functional on the latest versions of major desktop browsers (Chrome, Edge, Firefox).

NFR-04 (Security): All API routes (excluding login/register) shall require a valid JWT token to authorize data access and role-specific actions.

NFR-05 (Usability): The interface shall trigger a visual feedback element to confirm the success of a major interactive action, such as casting a vote.

5.5 Requirements prioritization and MVP boundary

The MVP scope is strictly defined by requirements marked as "Must" and "Should" (Quaility improvements for core features). Requirements marked as "Could" are included in the design to demonstrate the platform's potential; however, their full implementation is subject to the successful stabilization of the core features.

5.6 Traceability table

UC	FR	Related screens	Verification method
UC-01	FR-01, 02, 03	Login screen, Waiting page	Manual test of valid/invalid credentials and routing for users with no assigned group.
UC-02	FR-04,	Chat tab	Real-time broadcast test using two

	05, 06		parallel client sessions; verification of message metadata.
UC-03	FR-07, 08, 09	Announcement tab, Creation modal	Visual confirmation of chronological sorting and UI styling for urgent priority announcements.
UC-04	FR-10	Event creation form	Form submission test and check if displayed in schedule.
UC-05	FR-11, 12, 13	Polls tab	Attempting a double-vote to trigger restriction; verifying annoucement after poll expiration.
UC-06	FR-14, 15	Admin dashboard	Confirm action execution through a student/teacher user.
UC-07	FR-16, 17, 18	Schedule View	Visual test verifying color-coding logic and ensuring the details modal opens on click.
Global	FR-19, 20, 21, 22	Resources tab, General UI	File upload/download test; manual triggering of empty/invalid inputs to verify system rejection.

6. SYSTEM DESIGN

6.1 Design goals and constraints

The design of UniConnect is driven by the need for a responsive, real-time academic environment. The primary design goals include a desktop-first approach to maximize information density for study coordination and a modular architecture to separate business logic from data persistence. Key constraints from Chapter 4 dictate a "Vanilla" approach for the frontend and a containerized backend using FastAPI to ensure high performance and easy deployment via Docker.

6.2 User interface structure

The application uses a centralized navigation model driven by a persistent sidebar. Unauthenticated users are routed directly to login or registration. Post-authentication, unassigned users are directed to a holding screen, while assigned users land on the primary communication hub. From there, the sidebar enables seamless switching between functional modules without losing the global application state.

Authentication Screens (login.html, register.html): These serve as the secure entry points to the application. The user's primary action is to input their credentials to receive a JSON Web Token (JWT) or to create a new account, effectively acting as the gateway to the entire platform.

Unassigned State (no_groups.html): This view acts as a temporary holding area for newly registered users. Its sole purpose is to display a waiting message, restricting access to group-specific modules until a system administrator officially assigns the user to a class cohort.

Communication Hub (chat.html): This is the default landing view for active users. It facilitates real-time messaging, allowing students and professors to scroll through the group's message history and broadcast instant text updates to their class.

Announcements (announcements.html): This screen isolates priority communication. All users can consume official broadcasts, while authorized roles (Professors, Delegates) are provided with functional inputs to publish critical, visually distinct alerts.

Polls (polls.html): This view handles democratic decision-making. Users interact with this screen to view active elections, securely cast their votes, and experience the required multimedia success feedback upon completion.

Timetables (timetables.html): This screen serves as the academic scheduling dashboard. It allows users to view their official parsed calendar and provides forms to actively create and categorize custom study events.

Resources (resources.html): This view functions as the centralized file repository. It provides the necessary interface elements for users to browse, download, and securely upload categorized course materials.

Admin Dashboard (admin.html): This screen provides an exclusive, protected interface for system administration. It allows Teachers and Administrators to create new class cohorts and actively manage student enrollments via email lookup.

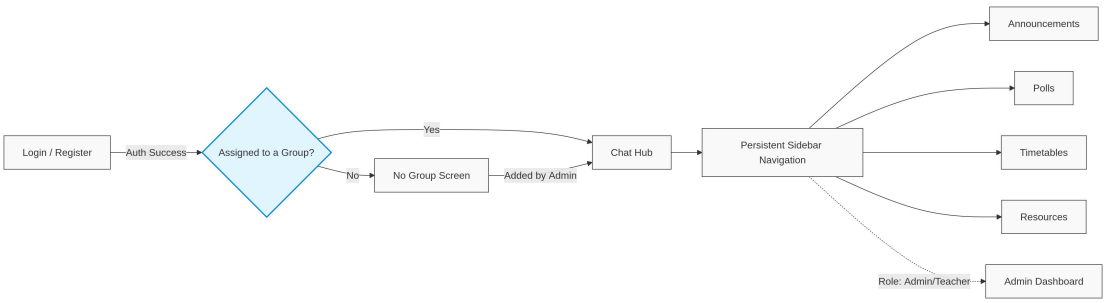

Figure 1: Application screen map and navigation flow.

6.3 Core user flows

The system design ensures primary use cases are reachable within two clicks. For UC-02 (Group Messaging), the flow begins with authentication; invalid credentials display an inline error, while successful logins fetch user data and redirect to either a holding page (if unassigned) or the main chat view. Once there, the chat.js script initializes a WebSocket handshake (/ws/chat/{group_id}) to dynamically inject incoming messages into the DOM without page reloads. If this connection fails, a red error banner is immediately displayed at the top of the screen.

6.4 System architecture and module design

UniConnect follows a classic Client-Server architecture with a specific focus on asynchronous communication.

Backend (FastAPI):

- Routers: Modular Python files handle specific REST API endpoints.
- Schemas: Modular Python files handle input validation.
- Core: Group of core components such as the global configuration of the server and the security related features.
- Services: Group of services needed to run the system such as the socket manager or the run of background tasks.
- Database: SQLAlchemy maps Python objects to the database.

Frontend (Vanilla JS):

- Page structure: Individual files (e.g., chat.html) structure each pages.
- Page style: Individual files (e.g., chat.css) styles each pages.
- Page Scripts: Individual files (e.g., chat.js) manage the specific logic and DOM manipulation for each view.
- Navigation side bar: Single .js file for handling page and group switch.
- Common (common.js): Handles shared logic (JWT, error handling, access rights, AppState).

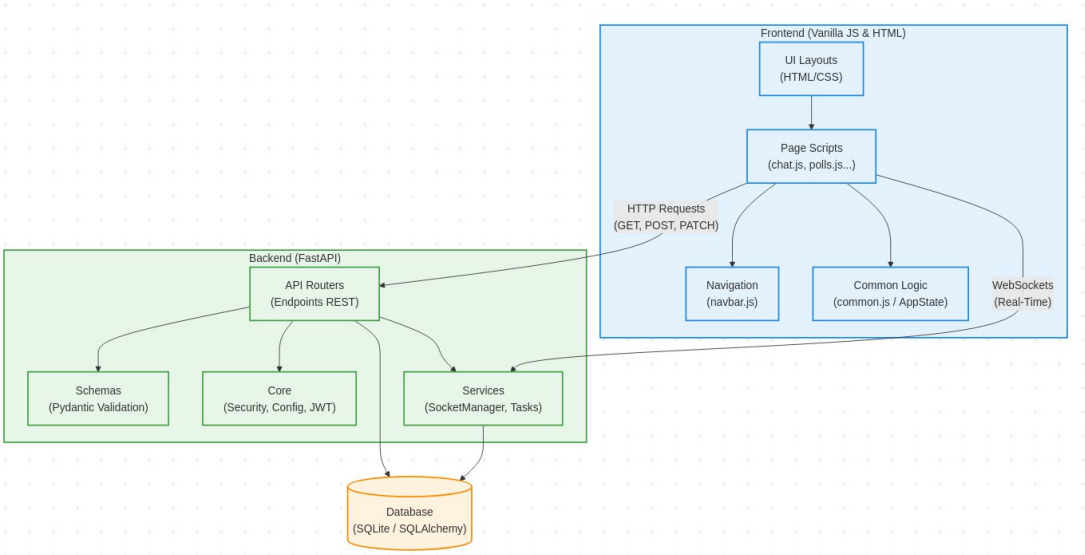

Figure 2: *UniConnect System Architecture and Component Dependencies.*

6.5 Data design

This subsection defines the internal data structure managed by the SQLAlchemy Object-Relational Mapper (ORM), specifically designed to maintain strict relational integrity across all academic features. All persistent data within the system is generated via direct user input and is securely stored in a local PostGres database, utilizing SQLAlchemy to seamlessly map Python classes to the corresponding database tables. Furthermore, physical files uploaded are saved directly to a local server

directory, with their specific access paths properly recorded within the database.

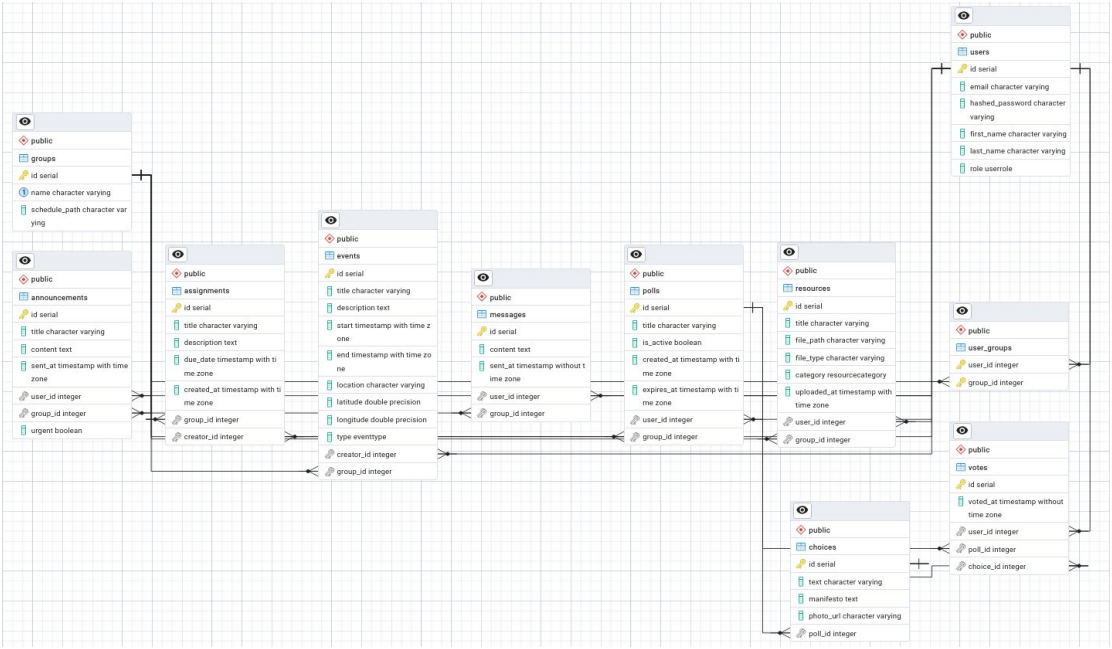

Figure 3: *Entity-Relationship Diagram (ERD) of the UniConnect database.*

6.6 Data flow and state management

UniConnect operates as a Single Page Application (SPA) where state is efficiently distributed between the Vanilla JS client and the FastAPI server. To ensure a fluid user experience, the client maintains a minimal local state. This primarily involves tracking the active authentication status via a securely stored JWT, managing the currently selected class group context, and handling transient UI conditions such as open modals, form inputs, or active error messages.

All significant state transitions are driven by asynchronous HTTP requests using the Fetch API, strictly adhering to RESTful principles. When retrieving data via GET requests, the client awaits the server's response before dynamically clearing and reconstructing the relevant DOM containers. For data creation (POST) and modification (PATCH), a successful server

response triggers immediate visual feedback—such as closing modals, clearing forms, or initiating CSS animations—while selectively refreshing the affected UI components. Similarly, successful DELETE requests result in the immediate extraction of the corresponding HTML element from the DOM without requiring a full page reload. Across all interactions, any failed requests instantly surface prominent error messages to guide the user.

Key Flow Example: Casting a Delegate Vote (UC-05)

1. User Action: The student clicks the "Submit Vote" button after selecting a candidate.
2. Client Request: The JS script intercepts the click, prevents default form submission, and sends a POST request to /api/votes with the poll_id, choice_id, and the user's JWT.
3. Server Operation: FastAPI receives the request, extracts the user ID from the JWT, and attempts to insert the Vote object. If the UniqueConstraint fails (user already voted), it returns a 403 Forbidden. If successful, it saves the vote and returns 200 OK.
4. UI Reflection: If 200 Success: The client disables the voting radio buttons (updating the local state), hides the "Submit" button, and triggers the multimedia CSS animation to confirm the action.

6.7 Multimedia design and integration

The application's multimedia elements consist of CSS-based animations specifically designed to provide immediate visual feedback and enhance the overall user experience. These animations are strategically triggered during key interactive moments that require clear confirmation. From a technical perspective, the system utilizes various visual effects achieved through CSS keyframes applied to dynamically generated DOM elements,

ensuring zero impact on initial page load times. Finally, while these animations may fail to render on significantly older web browsers, such legacy fallback scenarios are explicitly excluded from the scope of the current MVP.

6.8 Design decision justification

The most critical technical decision was the choice of the real-time communication protocol for the chat module.

Decision: WebSockets vs. HTTP Polling for Real-Time Interaction

Criteria	Weight	WebSockets	Long Polling
Real-time	30%	5/5 (Instant)	2/5 (Delayed)
Server resource usage	25%	4/5 (Low overhead)	1/5 (High overhead)
Implementation ease	20%	3/5 (Moderate)	4/5 (Simple)
Scalability	25%	5/5 (High)	2/5 (Low)
Weighted total	100%	4.35	2.15

WebSockets were chosen despite higher implementation complexity because they satisfy the strict performance requirements (NFR-02) and provide the seamless user experience expected in a modern messaging application.

6.9 Traceability summary

The modular architecture described in this chapter ensures that every requirement defined in Chapter 5 is explicitly owned by a dedicated system component, guaranteeing both traceability and maintainability through a

strict separation of concerns. On the backend, the core security and schema modules directly fulfill NFR-04 (JWT authorization) and FR-22 (strict user input validation). Simultaneously, backend services utilize a singleton socket_manager to handle real-time WebSocket broadcasting (FR-04, FR-06), while background tasks securely manage election lifecycles (FR-13). The API routers map precisely to the core MVP use cases (UC-01 to UC-07), supported by a SQLAlchemy database implementation that completely covers the required structural data constraints (DR-01 to DR-06). On the client side, the frontend common module's AppState singleton and global error-handling logic satisfy role verification (FR-02), unassigned user redirection (FR-03), and system reliability feedback (NFR-01). Finally, navigation scripts handle seamless group switching without page reloads (FR-15, FR-21), and individual page scripts execute the specific DOM manipulations required for usability features (NFR-05), such as triggering interactive success animations.

7. IMPLEMENTATION

7.1 Implementation environment and tools

The development of UniConnect was conducted across Linux and Windows using IDE, enhanced by extensions (Pyright, Prettier...). The backend architecture is powered by FastAPI (Python 3.12+), utilizing SQLAlchemy for ORM capabilities and Pydantic for data validation. This backend is served via Uvicorn and containerized using Docker. On the client side, the frontend interface was built without external frameworks, relying strictly on HTML5, JavaScript, and CSS. Multimedia elements use scalable SVG icons and pure CSS keyframe-based animations. Finally, the entire application is orchestrated through Docker Compose, enabling a one-command deployment of the complete environment.

7.2 Project structure

The repository follows a strict modular structure, ensuring a clear separation of concerns between the server-side logic and the client-side interface, as illustrated below:

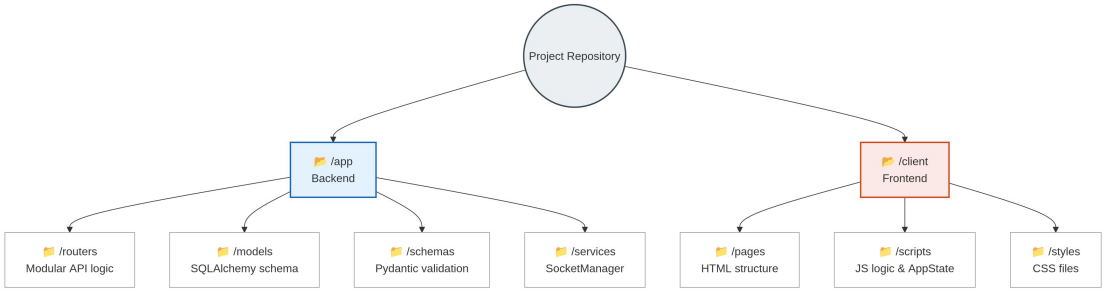

Figure 4: Project structure

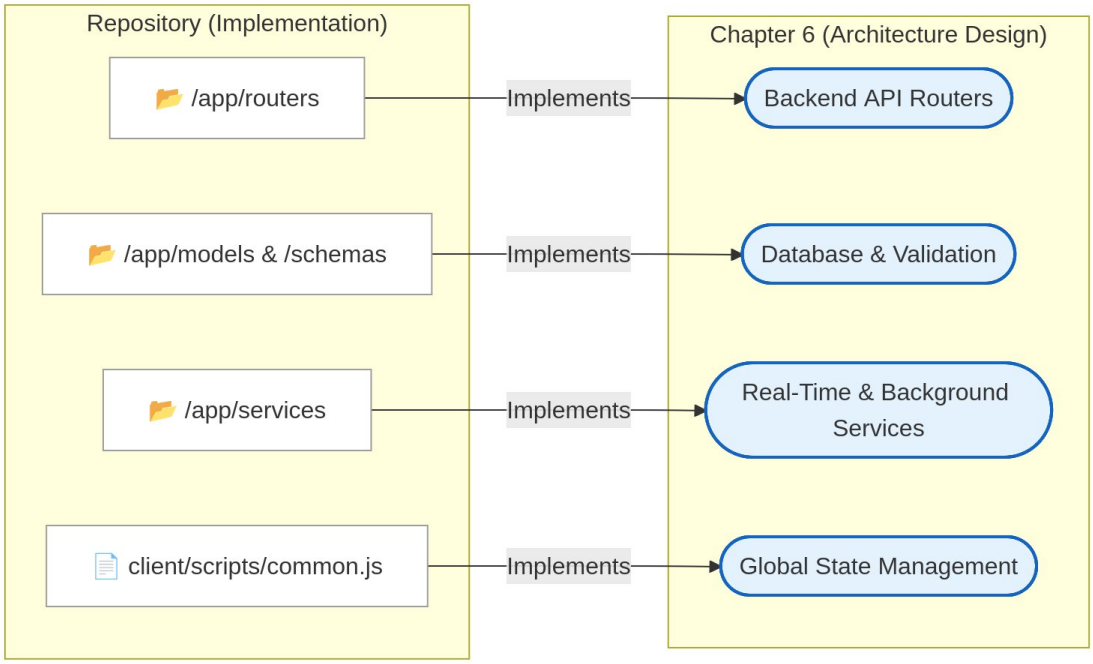

Figure 5: Traceability mapping between the physical repository and Chapter 6 architectural modules.

7.3 Implementation of screens/pages

The UniConnect frontend utilizes Vanilla JavaScript, where each screen manages its own DOM manipulation while relying on a shared AppState for global context. The Login and Register screens serve as secure entry points, handling credential inputs and seamless state toggling. Upon successful authentication, a JSON Web Token (JWT) is securely saved to localStorage, and the user is routed to the Home dashboard. Conversely, failed authentication attempts dynamically trigger a prominent red error banner.

The Chat screen (chat.html) acts as the real-time communication hub, featuring a scrollable history and instant DOM updates for new messages. It gracefully manages edge cases by displaying a "No messages yet" prompt for empty states and a red warning banner if the WebSocket connection drops, fulfilling requirement NFR-01. Alongside this, the Announcements

screen (announcements.html) isolates priority and normal updates into a detailed feed. Authorized users can open a creation modal, which, upon submission, triggers an asynchronous POST request and dynamically refreshes the feed.

To facilitate democratic decision-making, the Polls screen (polls.html) displays active elections via interactive radio buttons. Successfully casting a vote immediately transforms the UI to prevent duplication: inputs are disabled, the submit button is hidden, and a CSS confetti animation is triggered. The Timetables screen (timetables.html) handles academic scheduling by providing a comprehensive calendar view for official courses and custom study events, supported by event filtering and interactive creation forms.

Finally, the Resources screen (resources.html) functions as the centralized file repository, allowing users to download materials and submit new files, utilizing a guided empty state to prompt initial uploads. System administration is strictly isolated within the Admin Dashboard (admin.html), an exclusive interface where teachers and administrators can monitor class groups and manually enroll students via email lookup.

7.4 Implementation of core features

UC-01 (Role-Based Authentication): This use case details a secure role-based authentication system using OAuth2 and JWT. The client sends a POST request with credentials to `/auth/login`. The backend verifies the SQLite-stored hashed password and generates a JWT, which is saved in `localStorage` before redirecting to the dashboard. For error handling, incorrect passwords yield a 401 Unauthorized status, triggering a red error banner on the UI. Database connection failures return a 500 Internal Server Error, displaying a generic "Service unavailable" message. A key MVP limitation is

storing tokens in localStorage; production deployment requires HTTP-only cookies to mitigate XSS risks.

UC-02 (Real-Time Group Messaging): This use case describes a real-time messaging system powered by WebSockets. Upon loading the chat screen, chat.js establishes a connection to ws://.../chat/{group_id}. Sent messages are emitted to the backend SocketManager, saved to the database, and immediately broadcast to active group connections for direct DOM injection. For alternative flows, client-side JS validation silently blocks empty messages. If the network drops, the WebSocket onclose event triggers a red "Connection Lost" banner. A current limitation is the lack of lazy loading; the system only fetches the last 50 messages upon initialization.

UC-03 (Official Announcements): This use case implements a priority broadcasting system for announcements. When an authorized user submits the form, the client sends a POST request to /announcements. The backend persists the object, the client refreshes the feed, and the UI dynamically applies distinct CSS styling (like a red badge) to items flagged urgent=True. For error handling, HTML5 required attributes prevent submissions with blank titles, while API failures trigger a generic error toast. A notable limitation is the absence of push notifications; users must manually check the dashboard for updates.

UC-04/07 (Study Event & Timetable Management): This use case covers timetable management, featuring .ics parsing for official courses and custom event scheduling. Accessing the timetable prompts the client to fetch both parsed .ics data and custom database events to populate the calendar UI. Creating a custom session triggers a POST request to /events,

persisting the record and immediately refreshing the UI. For error handling, end times preceding start times trigger a 400 Bad Request and an "Invalid date range" UI alert. If the official .ics file is corrupted, the backend catches the parsing exception and degrades gracefully, showing only custom events alongside a UI warning. Limitations include the lack of recurring events and external calendar synchronization.

UC-05 (Vote System): This use case outlines a secure polling module. Submitting a vote sends a POST request containing the poll_id and choice_id. The backend creates a Vote record; upon a 200 OK response, the UI disables the form and injects .confetti DOM elements for a CSS keyframe animation. If a user maliciously attempts to vote twice via the API, the database's UniqueConstraint("user_id", "poll_id") fails, throwing an IntegrityError and returning a 403 Forbidden to trigger a "You have already voted" message. Server-side timestamp checks similarly reject votes for expired polls. A strict design limitation is complete vote anonymization, allowing administrators to view only total counts.

UC-06 (Administrator Dashboard): This use case defines the administrator dashboard for group management. To enroll a student, a Teacher submits an email address, triggering a POST request to the management endpoint. The backend queries the User table and inserts a record into the user_groups_association table. For alternative flows, non-existent emails return a 404 Not Found, prompting a "User not found" UI alert. Attempting to add an already enrolled user safely ignores the action and displays a "User already enrolled" badge. A defined MVP limitation is the lack of bulk CSV student uploads.

7.5 Data handling and persistence

For data management and persistence, the MVP utilizes a local SQLite database, thoughtfully designed with a seamless migration path to PostgreSQL via the database.py configuration to accommodate future scalability. The system guarantees immediate persistence for all critical interactions, ensuring that every chat message, poll result, and official announcement is instantly recorded. Furthermore, data integrity and backend security are strictly enforced by filtering all incoming payloads through Pydantic schemas prior to database insertion. This rigorous validation process effectively mitigates the risk of SQL injection and prevents the processing of malformed data. Finally, the resource module manages uploaded materials by automatically generating and assigning unique filenames within the server's local storage, thereby eliminating the risk of data overwrite or naming collisions.

7.6 Multimedia implementation

To provide immediate visual feedback, a "Confetti" success animation is triggered upon receiving a successful 200 OK response from the voting API. This feature is implemented as a highly optimized, lightweight CSS animation: once a vote is confirmed, the client-side script dynamically injects multiple div elements bearing the .confetti class into the DOM. These elements leverage pure CSS keyframes—specifically the transform: rotate() and translateY() properties—to visually simulate the physics of falling paper. From a performance perspective, relying exclusively on Vanilla CSS ensures that the rendering process does not block or impact the JavaScript event loop, thereby maintaining full user interface responsiveness throughout the visual sequence.

7.7 Key implementation decisions and architectural challenges

Navigating the architectural design phase proved to be the most significant challenge during implementation, necessitating several critical technical decisions. Primarily, rather than relying on a monolithic main.py structure, the backend API was deliberately separated into modular routers. Although this approach introduced initial design complexity, it ultimately streamlined debugging and significantly accelerated the development. On the frontend, adhering to a strict "no-framework" constraint without sacrificing modern aesthetics required the extensive application of CSS Flexbox and Grid paradigms. This strategy yielded a highly responsive user interface—particularly for the navigation sidebar and real-time chat feed—while entirely circumventing the performance overhead associated with heavy UI libraries like Bootstrap. Furthermore, to ensure state consistency across the client interface, an AppState singleton was implemented within the common.js module. By centralizing the tracking of the active class group, user role, and WebSocket connection status across all application tabs, this architectural pattern effectively resolved potential data desynchronization vulnerabilities.

7.8 Current status

The UniConnect MVP is fully implemented and operational. All "Must" and "Should" requirements from Chapter 5 are met. The "Could" requirement (Resource sharing) is also fully functional, allowing for a complete demonstration of the academic coordination concept.

8. TESTING PLAN AND RESULTS

8.1 Testing objectives and scope

The primary objective of this testing phase is to verify that the UniConnect Minimum Viable Product (MVP) operates reliably and meets all functional and non-functional requirements defined in Chapter 5.

The testing covers all primary Use Cases (UC-01 to UC-07), including the additionally implemented Resource module. Testing focuses on verifying the "Happy Path" (main success scenario) and handling alternative flows (user mistakes and system errors). The environment tested is desktop-first, utilizing the latest versions of Google Chrome, Microsoft Edge, and Mozilla Firefox. Mobile responsiveness testing is excluded due to the desktop-first design constraint. Automated load testing (e.g., simulating 10,000 concurrent users), email push notifications, and advanced penetration testing are excluded as they fall outside the MVP boundaries.

8.2 Testing approach and methodology

The testing strategy relies primarily on manual, use-case-based verification. Tests are executed manually against the deployed Docker container running the FastAPI backend and Vanilla JS frontend. A dual-testing approach was utilized where developers tested modules written by other team members to prevent confirmation bias. Upon fixing a defect, the entire use case flow was re-tested to ensure no new issues were introduced. A seeded database was utilized, pre-configured with 1 Administrator, 2 Teachers, 1 Delegate, 3 Students, and 2 active class groups to simulate a realistic academic environment.

8.3 Test cases derived from use cases

Each implemented Use Case was translated into actionable test cases covering the main success path and at least two alternative flows.

UC-01 (Authentication):

- TC-01A (Happy Path): User enters valid credentials; system issues JWT, saves to localStorage, and redirects to Dashboard.
- TC-01B (Alt - User Mistake): User enters an incorrect password; system prevents login and displays a red error banner (401 Unauthorized).
- TC-01C (Alt - System Issue): User attempts to register with an already existing email; API returns a 400 error, UI prompts "Email already in use."

UC-02 (Real-Time Chat):

- TC-02A (Happy Path): User sends a message; message is broadcasted via WebSocket and appears instantly on all connected clients.
- TC-02B (Alt - User Mistake): User attempts to send an empty message; client-side JS blocks the emission.
- TC-02C (Alt - System Issue): Server connection drops; UI immediately displays a red "Connection Lost" banner (NFR-01).

UC-03 (Official Announcements):

- TC-03A (Happy Path): Professor submits a new announcement flagged as "urgent"; the API saves the record, and the UI immediately updates the feed, applying the specific priority CSS styling (e.g., red badge) to the new item.
- TC-03B (Alt - User Mistake): Professor attempts to publish an announcement with an empty title; HTML5/client-side validation

intercepts the action and prevents the form submission, highlighting the required field.

- TC-03C (Alt - System Issue): The user's JWT token expires just before clicking submit; the API returns a 401 Unauthorized, and the UI redirects the user to the login page with a "Session expired" message.

UC-04 / UC-07 (Study Event & Timetable Management):

- TC-04A (Happy Path): Student creates a new custom study event with valid start and end dates; the backend saves the Event object, and the frontend dynamically renders it on the calendar view with the correct color-coding based on the EventType Enum.
- TC-04B (Alt - User Mistake): Student sets the event's "end date" to a time before the "start date"; the system (Pydantic schema validation) rejects the payload with a 422 Unprocessable Entity, and the UI displays "End date must be after start date."
- TC-04C (Alt - System/Data Issue): The official .ics schedule file for the group is missing or corrupted on the server; the backend catches the file parsing error safely, and the UI renders an empty calendar with a graceful "Official schedule currently unavailable" warning instead of crashing.

UC-05 (Vote System):

- TC-05A (Happy Path): Student selects a candidate and submits; UI disables buttons and triggers CSS confetti animation (NFR-05).
- TC-05B (Alt - User Mistake): User attempts to submit a vote without selecting a candidate; HTML5 validation prevents submission.

- TC-05C (Alt - System Issue): User attempts to bypass UI and double-vote via API; database UniqueConstraint rejects the entry, API returns 403 Forbidden, UI shows "Already voted."

UC-06 (Admin Panel):

- TC-06A (Happy Path): Teacher adds a student to a group using their email; student appears in the group roster.
- TC-06B (Alt - User Mistake): Teacher inputs a non-existent email; API returns 404, UI displays "User not found."

Resource Sharing System:

- TC-RES-A (Happy Path): User selects a valid PDF document and clicks upload; the backend securely saves the file to the local directory, records the path in the database, and the UI updates the list to show the new downloadable resource.
- TC-RES-B (Alt - User Mistake): User clicks the "Upload" button without attaching any file; the frontend validation catches the empty input and displays an "Please select a file to upload" alert.
- TC-RES-C (Alt - System/Data Issue): User attempts to download a file that has been accidentally deleted from the physical server disk (but the database record still exists); the FastAPI router returns a 404 Not Found, and the client UI catches this to display a "File not found or moved" error toast.

8.4 Test execution results

The following table summarizes the final regression test execution at the end of Sprint 3.

UC	TC	Scenario	Require	Result	Notes / Fix Reference

			ment		
01	01A	Login Success	FR-01	Pass	Valid JWT correctly saved to storage.
01	01B	Invalid Credentials	FR-02, FR-22	Pass	Handled natively by FastAPI OAuth2 logic.
01	01C	Duplicate Email Registratio n	FR-22	Pass	API returns 400 error; UI alerts user.
02	02A	Message Broadcast	FR-04, FR-06	Pass	WebSocket latency verified under 100ms.
02	02B	Empty Message Prevention	FR-22	Pass	Client-side JS and backend block emission successfully.
02	02C	WS Disconnect	NFR-01	Pass	Fixed by adding onclose JS event trigger.
03	03A	Create Urgent Announce ment	FR-07, FR-09	Pass	Priority CSS badge applies correctly on feed refresh.
03	03B	Empty Title Validation	FR-22	Pass	HTML5 validation effectively blocks submission.
03	03C	Expired JWT Token	NFR-04	Pass	UI catches 401 Unauthorized and redirects to login.
04 07	04A	Create Custom	FR-10, FR-16	Pass	Event renders correctly on the Timetable view.

		Event			
04 07	04B	Invalid Date Range	FR-22	Pass	Backend Schema rejects payload with 422 error.
04 07	04C	Missing .ics File	NFR-01	Pass	Backend parses safely; UI renders graceful warning.
05	05A	Cast Vote	FR-12, NFR-05	Pass	Confetti animation triggers without UI lag.
05	TC-05B	Empty Vote Submission	FR-22	Pass	HTML5 validation prevents unselected submission.
05	05C	Prevent Double Vote	FR-12	Pass	DB UniqueConstraint successfully rejects duplicate.
06	06A	Add User to Group	FR-14	Pass	Association table updates correctly in SQLite.
06	06B	Add Unknown User	FR-14	Pass	UI correctly catches 404 Not Found error.
RES	RES A	Upload PDF Document	FR-19, FR-20	Pass	Files save to local disk with unique identifiers.
RES	RES-B	Empty File Upload	FR-22	Pass	Frontend successfully blocks empty attachment.
RES	RES-C	Handle Missing File	NFR-01	Pass	Graceful failure if file is missing from server disk.

8.5 Defects and fixes

During the testing cycles, several bugs were discovered and subsequently resolved to stabilize the MVP.

Defect 01, classified as a major demo-blocking issue, occurred when users switched between groups via the sidebar; the chat history from the previous group remained on the screen, and new messages were inadvertently sent to the wrong WebSocket room. The root cause was identified as the Vanilla JavaScript DOM failing to clear previous elements, coupled with the old WebSocket instance remaining open. To resolve this, a global cleanup function was implemented within the AppState singleton in common.js, which explicitly calls .close() on active sockets and clears the .innerHTML of the chat container before fetching new group data, resulting in a successful re-test.

Defect 02 was a major severity issue involving a race condition where a user could bypass the single-vote restriction by rapidly clicking the "Submit Vote" button multiple times. The root cause stemmed from the client-side button disabling mechanism executing too slowly, allowing multiple API requests to be dispatched before the initial one completed. The applied fix enforced a strict SQLAlchemy UniqueConstraint("user_id", "poll_id") at the database level. Consequently, even if multiple simultaneous requests arrive from the client, the database actively rejects duplicates by throwing an IntegrityError, ensuring absolute data validity and successfully passing subsequent re-tests.

Finally, Defect 03 was a minor issue where uploading a resource with the exact same filename as an existing file inadvertently overwrote the older file on the server's local disk. This was caused by a lack of unique file naming logic within the resource.py backend router. The development team fixed this by modifying the backend logic to automatically append a unique UUID

or timestamp to all filenames prior to saving them to the disk, resolving the issue upon re-testing. Alongside these primary defects, several other minor, undocumented UI and logic bugs were similarly addressed throughout the testing phase to further stabilize the platform.

8.6 Non-functional checks

The interface effectively guides users. Important destructive or definitive actions (like voting) offer immediate visual feedback (Confetti animation, button state changes). Empty states (e.g., "No resources shared yet") are correctly implemented across all modules. Under normal network conditions, REST API requests (GET, POST) complete in under 200ms. WebSocket message transmission is virtually instantaneous, comfortably meeting the <500ms constraint. The application’s CSS Grid and Flexbox layouts render perfectly on Chrome, Edge, and Firefox without requiring vendor prefixes. Accessing any secured router endpoint (e.g., /api/groups) without a valid Bearer token consistently yields a 401 response.

8.7 Limitations and risks

While the MVP successfully fulfills its core objectives, some technical limitations remain. The current chat implementation fetches the entire message history (or a hardcoded limit) on load. As a group's history grows massive over a semester, this could impact initial load times. Lazy-loading/pagination is required for a production release. Tokens are currently stored in localStorage for ease of implementation in Vanilla JS. While acceptable for an MVP, this exposes the tokens to Cross-Site Scripting (XSS) risks. A future iteration should migrate to HttpOnly cookies.

RESULTS AND CONCLUSIONS

Goal restatement

The primary goal of this project was to design and develop the Minimum Viable Product (MVP) of UniConnect: a centralized, real-time academic coordination platform aimed at streamlining communication, scheduling, and democratic decision-making within university class groups. By providing an integrated environment for students, delegates, and professors, the project sought to resolve the fragmentation of academic tools.

Task-by-task completion summary

The project was executed by systematically addressing the operational tasks defined in Chapter 3. First, the system architecture and database schema were successfully designed and implemented. The foundation was laid using FastAPI and SQLAlchemy to ensure a robust backend, while the frontend was constructed using Vanilla JavaScript and CSS. Second, the core real-time communication module was delivered successfully. The WebSocket-based chat system allows instant messaging within isolated class groups, with immediate persistence to the database. Third, the specific academic coordination tools were implemented. The polling system securely registers anonymous votes, the announcement module correctly prioritizes urgent broadcasts, and the timetable view successfully renders custom study events. Fourth, the required multimedia element was integrated as a lightweight CSS keyframe animation that triggers upon a successful vote submission, enhancing user feedback. Finally, the system was fully containerized using Docker, and rigorous use-case testing was conducted to verify both main success paths and alternative flows.

Results summary

The final delivered system is a fully functional web application that supports all defining MVP use cases end-to-end. Users can securely authenticate via JWT (UC-01), communicate in real-time (UC-02), broadcast priority announcements (UC-03), schedule events (UC-04/07), cast secure votes (UC-05), and manage group rosters (UC-06). Furthermore, the Resource Sharing System, initially scoped as a "Could-have," was successfully implemented, allowing file uploads and downloads. All interactions are persistently stored in a relational SQLite database. The multimedia aspect actively contributes to the user experience by providing psychological closure in the polling flow, without compromising the application's performance.

Testing outcome summary

System verification demonstrated a high degree of reliability. As documented in the testing phase, 100% of the MVP use cases passed their acceptance criteria, covering both the happy paths and required alternative flows (such as error handling and data constraints). The most critical defect resolved during testing was a WebSocket room desynchronization issue that occurred when switching between class groups, which was fixed by enforcing strict socket cleanup routines in the client state manager. The most notable remaining limitation is the lack of pagination in the chat history, which loads the entire sequence of messages upon connection.

Personal conclusions Developing UniConnect from scratch provided profound insights into full-stack software engineering. The decision to use modular FastAPI routers and Pydantic schemas proved to be highly effective, as it made the backend predictable, self-documenting, and easy to debug. Similarly, relying on database-level constraints was a correct

choice that elegantly prevented complex race conditions. Conversely, building a Single Page Application (SPA) purely with Vanilla JavaScript was more challenging than anticipated. While it successfully eliminated framework overhead, manually managing the DOM and synchronizing the global AppState across different views was time-consuming and prone to desynchronization bugs early in development. To deliver a stable product within the deadline features were cut from the scope to guarantee that the core mechanics were flawless.

Future work

If the project were to be developed beyond the MVP phase, the following logical next steps would be pursued:

1. Chat Pagination: Implementing lazy-loading for the chat interface to maintain performance as message histories grow over a semester.
2. Database Migration: Transitioning the database from SQLite to PostgreSQL to handle high concurrency during large-scale elections.
3. Institutional SSO: Integrating the authentication system with existing university infrastructure (e.g., LDAP, OAuth2 via Microsoft/Google) for seamless student onboarding.
4. Push Notifications: Adding Web Push API or email integrations to alert users of urgent announcements even when the application is closed.
5. Progressive Web App (PWA): Enhancing the frontend to serve as a PWA, granting users a native-like mobile experience with offline capabilities.

LIST OF REFERENCES

- Docker Inc. (2024). *Docker Documentation: Docker Compose*. Retrieved from https://docs.docker.com/compose/
- FastAPI. (2024). *FastAPI framework, high performance, easy to learn, fast to code, ready for production*. Tiangolo. Retrieved from https://fastapi.tiangolo.com/
- Fette, I., & Melnikov, A. (2011). *The WebSocket Protocol* (RFC 6455). Internet Engineering Task Force (IETF). Retrieved from https://datatracker.ietf.org/doc/html/rfc6455
- Flanagan, D. (2020). *JavaScript: The Definitive Guide: Master the World's Most-Used Programming Language* (7th ed.). O'Reilly Media.
- Mozilla Developer Network [MDN]. (2024). *CSS Layout: Grid and Flexbox*. Web Docs. Retrieved from https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout
- Fonticons, Inc. (2024). *Font Awesome Icons. Retrieved from https://fontawesome.com/*
- Google. (2024). *Material Design 3. Retrieved from https://m3.material.io/*
- Google Fonts. (2024). *Material Symbols & Icons. Retrieved from https://fonts.google.com/icons*
- Refactoring UI. (2024). *Heroicons. Retrieved from https://heroicons.com/*
- Ries, E. (2011). *The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses*. Crown Business.
- SQLAlchemy Authors. (2024). *SQLAlchemy 2.0 Documentation: The Database Toolkit for Python*. Retrieved from https://docs.sqlalchemy.org/